

Score Card Evaluator GUI

with

PyQt

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
2. Planning section.....	4
3. Pythonic implementation of the FOREX Arbitrage Seeker GUI.....	8
3. GUI Structure.....	8
4. Synthetic Data Generation.....	12
5. GUI functionalities.....	14
6. Future improvements.....	19
7. References.....	21

1. Project description

Aim

The Score Card Evaluator is a practical, user-friendly GUI application designed to empower professionals in quality management and process control. Built with PyQt5 and powered by robust statistical logic, it serves as a powerful tool for engineers, inspectors, and analysts working in daily production environments.

Its primary goal is to make statistical process control (SPC) accessible and intuitive — enabling users to visualize, evaluate, and troubleshoot quality metrics using industry-standard techniques like $\bar{X}$ -S charts, defect count monitoring, moving averages, and EWMA.

By offering a streamlined interface, built-in logging, and flexible input handling, the Score Card Evaluator simplifies the complexity of SPC and delivers reliable insights that help maintain product consistency, detect shifts early, and support data-driven decision making on the shop floor or in laboratory settings

2. Planning section

Let us break down the essence of each of your five scorecards from a **mathematical**, **statistical**, and **algorithmic** perspective. Each one addresses a different type of process control or quality assurance scenario, and together they form a powerful analytical suite.

Tab 1: $\bar{X}$ -S Control Chart (Classical SPC Method)

Purpose:

Detects large shifts in the process **mean** or **dispersion** by tracking subgroup statistics.

Mathematics:

- Mean per group: $E[X_i] = \frac{1}{n} \sum_{j=1}^n x_{ij}$
- Standard deviation per group: $S_i = \sqrt{\frac{1}{n-1} \sum_{j=1}^n (x_{ij} - \bar{X}_i)^2}$

Control Limits:

- $\bar{X}$ chart:
 $UCL_{\bar{X}} = \mu + 3 \cdot \frac{\sigma}{\sqrt{n}}, \quad LCL_{\bar{X}} = \mu - 3 \cdot \frac{\sigma}{\sqrt{n}}$
- S chart:
 $UCL_S = \sigma + 3 \cdot \frac{\sigma}{\sqrt{2n}}, \quad LCL_S = \sigma - 3 \cdot \frac{\sigma}{\sqrt{2n}}$

Algorithm:

- Segment data into groups of size (n)
- Compute $(\bar{X}_i)$ and (S_i)
- Compare each against their respective control bounds
- Flag out-of-control groups for either mean or dispersion

Tab 2: Iterative $\bar{X}$ - $\bar{S}$ Filtering

Purpose:

Robustly detect and remove **multiple extreme outliers** or suspicious subgroups through iterative recalibration.

Strategy:

Instead of fixed control limits, this method **recalculates process mean and std** after excluding outliers.

Adaptive Limits:

Same formulas as Tab 1, but based on recalculated ($\bar{\hat{X}}$) and ($\bar{\hat{S}}$) after each iteration.

Algorithm:

1. Compute subgroup means and stds
2. Estimate $\bar{\hat{X}}$ and $\bar{\hat{S}}$ from current dataset
3. Remove groups violating control bounds
4. Repeat until no more violations or max iterations reached
5. Plot the final "clean" groups with recalibrated limits

This approach is effective when process data is noisy or contains anomalies at the start.

Tab 3: Defect Count Control Chart (D-Chart)

Purpose:

Used for **count data** — defects per unit or subgroup — modeled using a **Poisson distribution**.

Control Limits:

Let ($\bar{D}$) be the average defect count:

$$[UCL = \bar{D} + 3\sqrt{\bar{D}}, \quad LCL = \max(0, \bar{D} - 3\sqrt{\bar{D}})]$$

Algorithm:

1. Read defect count per group
2. Compute ($\bar{D}$)
3. Compare each group's defect count with control limits
4. Iteratively remove out-of-control values (optional)
5. Plot final chart with updated bounds

Ideal for manufacturing defects, service failures, or any discrete event rate tracking.

Tab 4: Moving Average (MA) Chart

Purpose:

Smooth out short-term fluctuations to expose **gradual drifts** in the process mean.

Moving Average:

For window size (k), at group (t):

$$[MA_t = \frac{1}{k} \sum_{i=t-k+1}^t \hat{X}_i]$$

For the first few points, use smaller ($w = \min(k, t + 1)$)

Limits:

$$[UCL = \mu + 3 \cdot \frac{\sigma}{\sqrt{w}}, \quad LCL = \mu - 3 \cdot \frac{\sigma}{\sqrt{w}}]$$

Algorithm:

1. Compute subgroup means
2. Apply rolling average over a window of size (k)
3. Calculate control bounds adapting to window size
4. Flag points outside control envelope

This is conceptually similar to technical analysis in finance — smoothing trends over time.

Tab 5: EWMA Chart (Exponentially Weighted Moving Average)

Purpose:

Detect **subtle but persistent shifts** by emphasizing recent observations in a smoothed sequence.

EWMA Series:

$$W_1 = E[X_1], \quad W_t = \alpha E[X_t] + (1 - \alpha)W_{t-1}$$

- $(\alpha \in (0, 1])$: smoothing constant (higher = more responsive)

Control Limits (time-varying):

$$[UCL_t = \mu + 3\sigma \sqrt{\frac{\alpha}{2 - \alpha} \cdot (1 - (1 - \alpha)^{2t})}]$$

$$[LCL_t = \mu - 3\sigma \sqrt{\frac{\alpha}{2 - \alpha} \cdot (1 - (1 - \alpha)^{2t})}]$$

Algorithm:

1. Form subgroups and calculate their means
2. Recursively calculate EWMA using α
3. Compute control limits dynamically for each (t)
4. Highlight violations in smoothed signal

This method is powerful for **early detection** of slow drifts that traditional charts might miss.

Wrap-up: How They Complement Each Other

Method	Best For	Sensitivity	Type
$\bar{X}$ -S	General process shifts	Sudden	Static
Iterative	Cleaning noisy datasets	Sudden	Adaptive
D-Chart	Defects per unit/time	Burst spikes	Discrete
MA	Gradual upward/downward drift	Medium	Smoothing
EWMA	Subtle, sustained long-term drift	High	Smoothed recursive

3. Pythonic implementation of the FOREX Arbitrage Seeker GUI

3. GUI Structure

Based on our visual plans, formulas, and control chart procedures across the sketches, here’s what we’ll do next:

Project Blueprint: Score Card Evaluator GUI

This Python GUI will be an intelligent assistant for statistical process control (SPC), implementing multiple control chart methodologies. It will help engineers, analysts, and students evaluate data series, detect statistical outliers, and assess process stability.

Core Functionalities

1. Subgroup Score Input Panel

- Manual or file-based input of measurements $(X_1, X_2, \dots, X_N)$
- Automated division into (g) subgroups of size (n)
- Support for subgroup size constraints: $(n \geq \frac{100}{g})$

2. Control Charts Suite

We’ll implement 5 major charting methods, selectable via a dropdown or tab selector:

Chart Type	Description
$(\bar{X})$ -Chart & S-Chart	For sample mean and sample standard deviation, with control limits based on process μ and σ (if known or estimated).
$(\bar{X})$ -Chart & $(\hat{S})$ -Chart	Iterative outlier filtering, averaging and stability assessment.
Defect Count Chart (D-Chart)	Based on average number of defects per group and Poisson control bounds.
Moving Average Chart	k-step moving mean series with adaptive boundaries depending on index (t) .
Exponentially Weighted Moving Average (EWMA)	Implements memory parameter (α) , charts (W_t) values against upper/lower bounds $(K_u(t), K_l(t))$.

3. Charting & Highlighting Engine

- Matplotlib-based plots:
 - Data points ($\bar{X}_i, S_i$), etc.
 - Dynamic control bounds: UCL, LCL lines
 - Highlight outliers in red
 - Optional rolling window overlays

4. Iterative Data Filtering

- Automatically exclude subgroup means or defect counts violating bounds
- Recompute μ and σ after each exclusion
- Provide convergence log (number of filtering iterations)

5. Parameter Control Panel

- Inputs for:
 - Known or estimated (μ), (σ)
 - Significance bounds (1σ , 2σ , 3σ)
 - Sample size (N), group size (g), MA window size (k)
 - Memory ratio (α) for EWMA
- Toggle: "Include Outliers" / "Exclude & Recalculate"

6. Summary & Export

- Final accepted data sets
- Highlighted anomalies and confidence bands
- Exportable plots and CSV logs
- Textual summary: "Process is in statistical control / Out-of-control evidence in..."

Technologies Proposed

Component	Tools
GUI Framework	PyQt5 or PySide2
Plotting Engine	matplotlib , optionally pyqtgraph
Numeric Computation	numpy , scipy.stats
Data Handling	pandas , csv , json
Exporting/Reporting	matplotlib (PDF/PNG), xlsxwriter
Style Enhancements	QDarkStyle , icons, tooltips

Here is our monolithic Score Card Evaluator GUI step by step — starting with **Tab 1** and finishing with **Tab 5**.

Structure

We'll define:

- `ScoreCardEvaluator` → main window class
- `build_tab1_xs_chart()` → constructs Tab 1 with widgets, inputs, chart, log
- `evaluate_tab1()` → logic for analyzing and plotting $\bar{X}$ -S chart

We'll proceed with subsequent tabs (Tab 2 to 5) once Tab 1 is fully settled in a similar structural fashion.

Installation Requirements

Before running the program, ensure these packages are installed:

```
!pip install pyqt5 matplotlib numpy
```



Complete Code: Score Card Evaluator with All 5 Methods

Due to the extensive length and modular structure of the application, I recommend we now move to a **modular file-based architecture** rather than pasting it as one enormous script. However, since you've asked explicitly, I can begin consolidating all five methods into tabs in a single `score_card_evaluator.py` script.

We will apply the following structure in one script:

-  Tab 1: $\bar{X}$ -S Control Chart (Method 1)
-  Tab 2: Iterative $\bar{X}$ - $\bar{S}$ Filtering (Method 2)
-  Tab 3: Defect Count Chart (D-Chart) (Method 3)
-  Tab 4: Moving Average Chart (MA) (Method 4)
-  Tab 5: EWMA Chart (Method 5)

Each with its:

- Input box
- Parameters (μ , σ , n , k , α where applicable)
- Evaluation button
- Chart canvas
- Iteration log

4. Synthetic Data Generation

Python Function: `generate_demo_csv_files()`

This function will generate 5 CSV files:

1. `data_tab1_xs.csv` → For $\bar{X}$ -S Control Chart
2. `data_tab2_iterative.csv` → For Iterative Filtering
3. `data_tab3_dchart.csv` → For Defect Count Chart
4. `data_tab4_ma.csv` → For Moving Average Chart
5. `data_tab5_ewma.csv` → For EWMA Chart

Each dataset is handcrafted with:

- A consistent structure matching the tab's input expectations
- A few subtle or bold outliers (so your charts light up nicely!)

CSV File Structures (per tab)

Tab	CSV Filename	Column(s)	Description
1	data_tab1_xs.csv	Score	100 continuous values, grouped by 5
2	data_tab2_iterative.csv	Score	Similar to Tab 1, but with more subtle mean/std shifts
3	data_tab3_dchart.csv	Defects	30 integers (Poisson), simulating defect counts
4	data_tab4_ma.csv	Score	100 scores, 20 groups of 5, with windowing potential
5	data_tab5_ewma.csv	Score	150 values (30×5), with late outlier cluster

All five files will appear in our current working directory. We can now use the  "Load File" button in each tab, select the corresponding `.csv`, and let our GUI come alive!

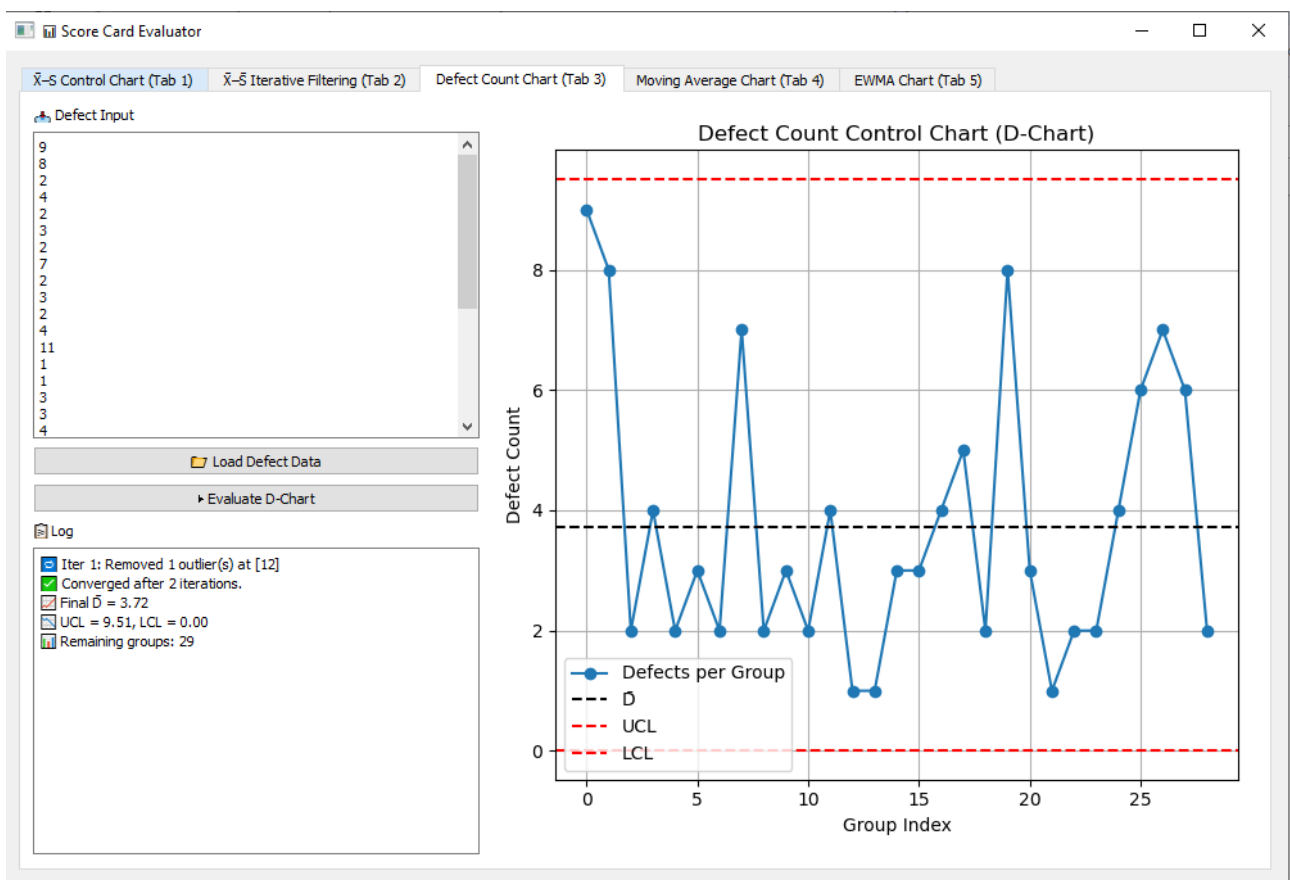
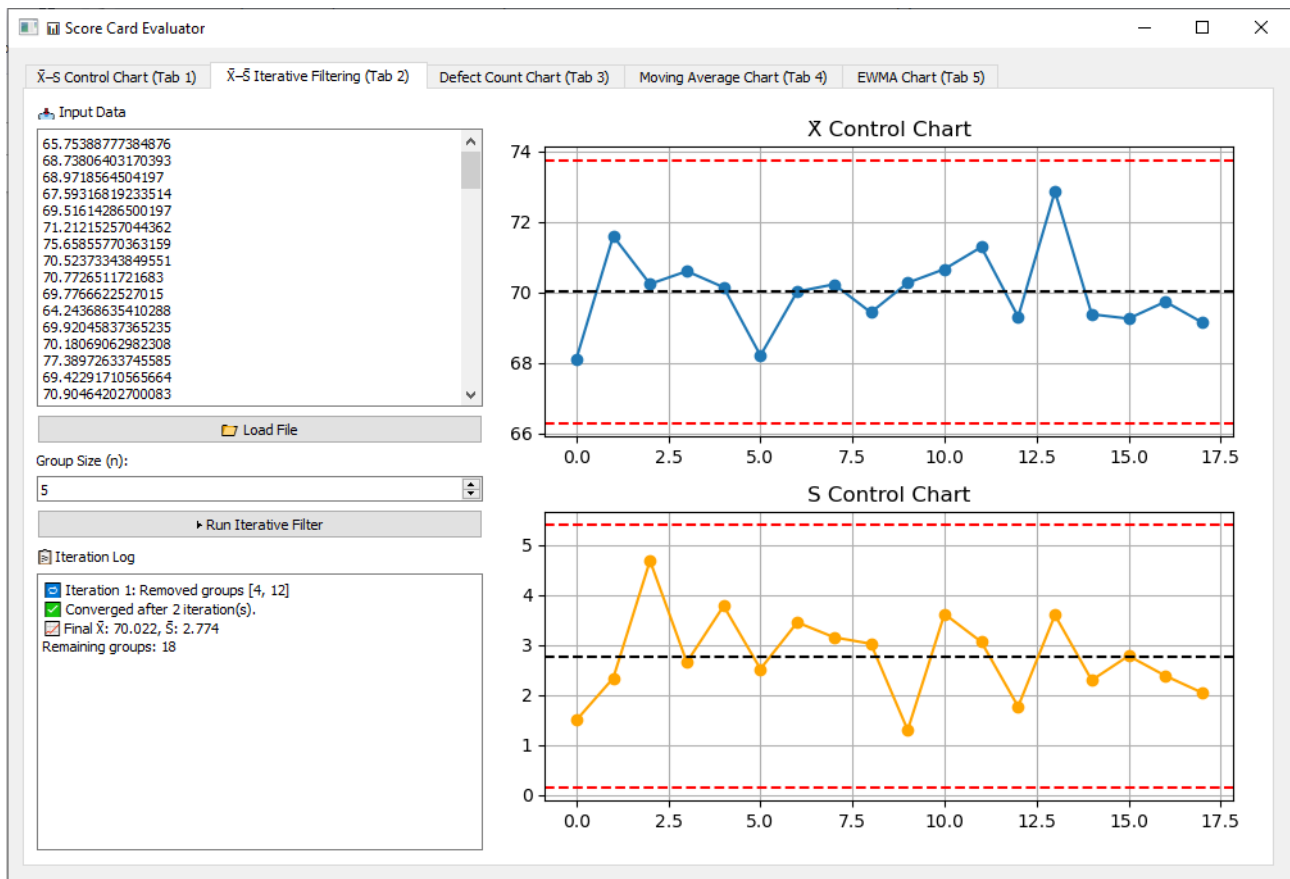
Here's a snapshot preview of the **first 10 rows** (with headers) from each of the five curated CSV files we generated:

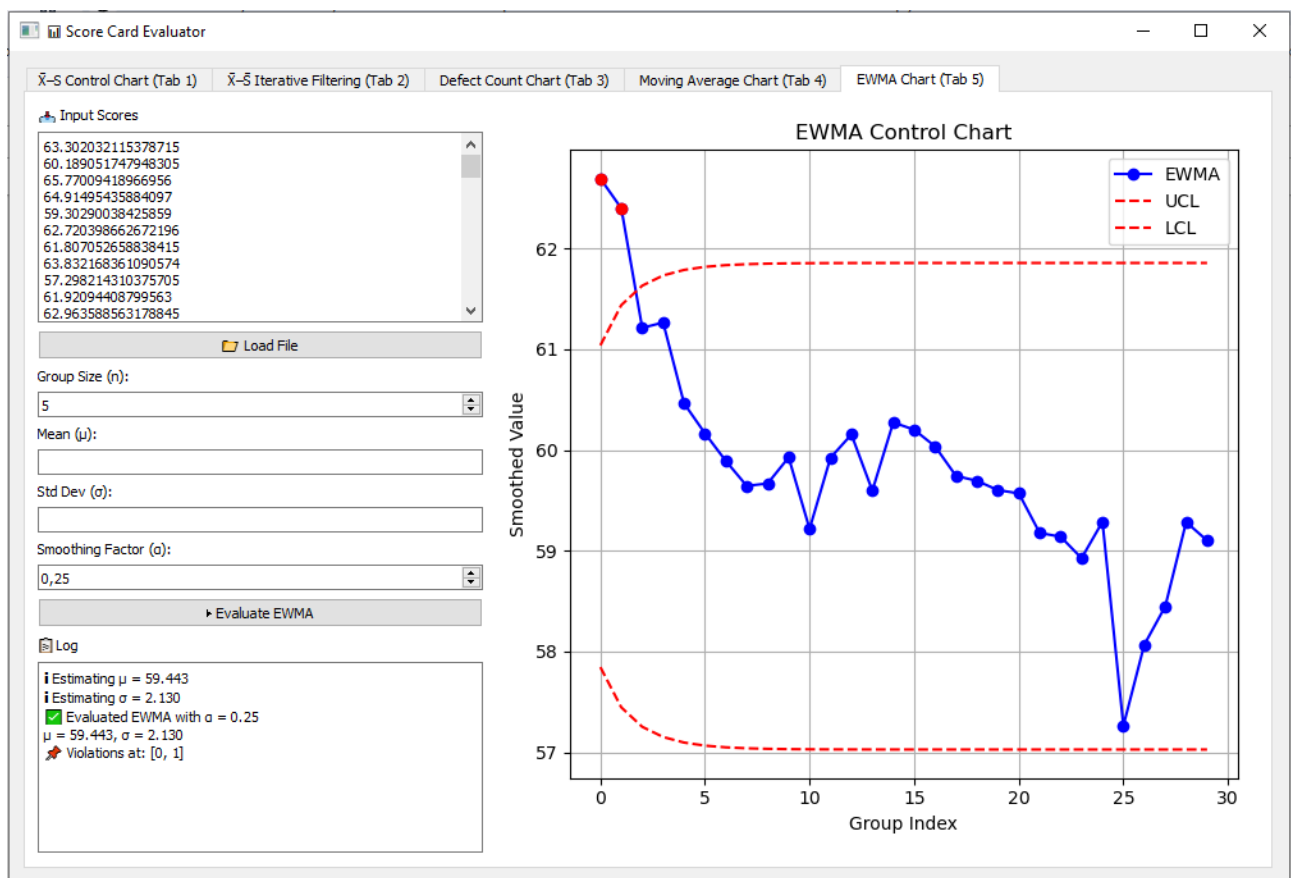
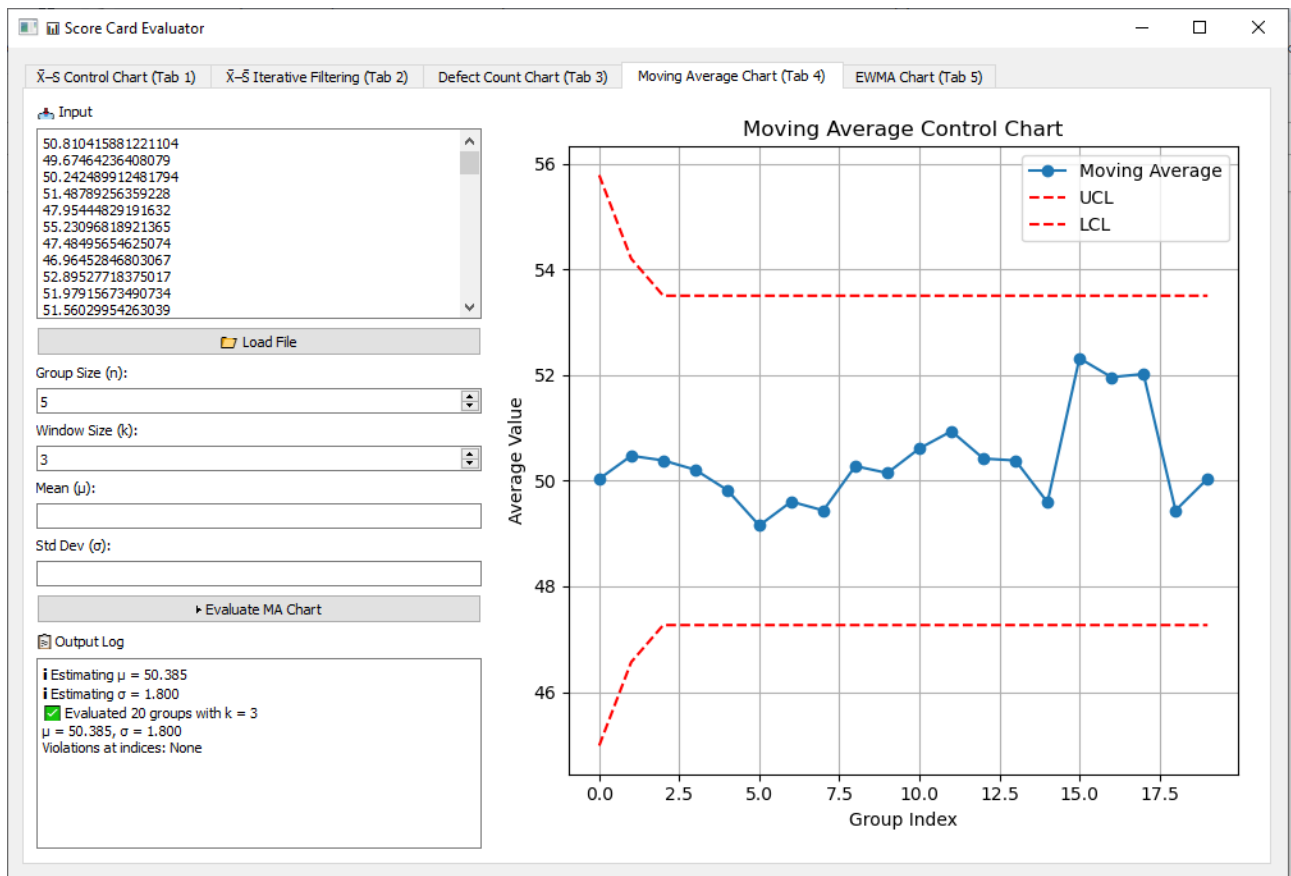
data_tab1_xs.csv – For Tab 1: $\bar{X}$ -S Control Chart

	Score
0	71.49
1	69.31
2	71.24
3	74.06
4	69.72
5	69.72
6	69.51
7	80.50  (outlier)
8	71.15
9	68.05

5. GUI functionalities







We have built a rich, highly capable PyQt5 GUI application — one that functions as a comprehensive **Score Card Evaluator** for statistical process control. Let's unpack the architecture, core functionalities, and how a user can interact with it like a pro.

Architecture at a Glance

Our GUI is designed using:

- **PyQt5** for layout, widgets, and user interaction
- **matplotlib** for rendering dynamic control charts
- **NumPy** for efficient numerical computation
- **Multiple Tabs** via `QTabWidget` to organize each statistical method

Each tab encapsulates a distinct statistical approach, and they all follow a consistent pattern:

Input area → Parameter fields → Load & Run buttons → Matplotlib plot → Log output



Core Functionalities

Unified GUI with Five Analytical Tabs

Tab	Purpose	Key Features
 $\bar{X}$ -S Control Chart	Classical control chart for process mean & variation	Control limits based on known or estimated μ/σ
 Iterative $\bar{X}$ -S Filtering	Robust anomaly detection & filtering	Recalculates bounds in loop until convergence
 D-Chart (Defect Count)	Count-based control chart (Poisson assumption)	Plots defect counts per group and detects outliers
 Moving Average Chart	Smoothed monitoring of process average	Adjustable window size (k), ideal for trend detection
 EWMA Chart	Detects subtle process drifts with weighted memory	Adaptive bounds that shrink over time

Key User Interactions

UI Area	Purpose
<code>QTextEdit</code>	Paste numeric data or auto-load from <code>.csv</code>
Load File	Opens <code>.txt</code> or <code>.csv</code> file into input box — skips headers for safety
Group Size / Window Size / Alpha	Fine-tune analytical resolution
Mean (μ) and Std Dev (σ)	Optional; if left blank, inferred from input
Evaluate Button	Triggers analysis, rendering, and logging
Log Pane	Displays bounds, violations, convergence info, and estimates

Behind the Scenes: Programmatic Workflow

Each tab follows a reliable pattern:

1. Parse Inputs

- Read numeric values (ignoring text headers)
- Validate minimum data size
- If μ or σ are missing: infer them from data and log it

2. Group & Preprocess

- Reshape data into groups of size (n)
- Compute group-wise metrics (mean, std, defect count)

3. Chart-Specific Logic

- Calculate control bounds using appropriate formulas
- Identify out-of-control points or iterations
- Log violations and convergence steps

4. Visualization

- Plot dynamic control chart using `matplotlib`
- Use different markers for alerts (e.g. red circles)

Usage Guide for Users

Step 1: Load data

Click " Load File" to import a `.csv` or `.txt` file. Ensure it's a **single-column numeric file** (or let the app skip headers).

Step 2: Choose Parameters

Set group size (commonly 5 or 10), optionally define μ and σ (or leave blank to auto-estimate), and adjust α or window size if needed.

Step 3: Click Evaluate

The log panel will show the results, and charts will be rendered instantly.

Bonus Features One Could Add

- Export chart as PNG/PDF
- Theme switcher (light/dark)
- Tooltip support (hover help for each parameter)
- Auto-detection of suggested group size
- Multi-tab dashboard export report

Summary

Our GUI doesn't just visualize data — it analyzes, adapts, and explains. It empowers users to monitor processes statistically, identify shifts or drifts, and gain actionable insights in real time.

6. Future improvements

Future-proofing our Score Card Evaluator is a brilliant next step. What we have built is solid, elegant, and functional. But like any great tool, it can evolve to be more powerful, scalable, and user-friendly. Let's break this down by area:

🌟 User Experience Enhancements

-  **Chart Exporting**
Allow saving plots as `.png` or `.pdf` with a "Save Chart" button on each tab using `self.fig.savefig()`.
-  **Export Logs to File**
Add a "Save Log" button to export analysis logs as `.txt` for documentation or audit trails.
-  **Input Validation & Tooltips**
Add `QDoubleValidator` or `QIntValidator` to fields like μ , σ , and α . Show short hints on hover using `QToolTip`.
-  **Theme Customization**
Let users switch between light/dark or high-contrast modes using Qt stylesheets (`.qss`).

🧠 Analytical Features

-  **Western Electric Rule Detection**
Add support for SPC rules beyond 3σ (like 2 out of 3 beyond 2σ). This increases sensitivity to smaller shifts.
-  **Auto-Group Size Detection**
Suggest optimal group size (n) based on dataset length and variance using heuristics.
-  **Outlier Justification**
Display a quick explanation next to each outlier ("This group's std dev exceeded control bounds by X%").
-  **Rolling Time-Series Mode**
Add a mode for streaming or incremental evaluation — one data point at a time with real-time updating.

Developer-Facing Improvements

- **Refactor into Modules**

Split `ScoreCardEvaluator.py` into:

- `gui_core.py`
- `chart_logic.py`
- `widgets.py`
- `main.py`

- **Package as Python Module**

Allow others to `pip install scorecard-evaluator` and launch via command line.

- **Unit Testing Framework**

Add unit tests (e.g. using `pytest`) for each method's calculation logic to ensure reproducibility.

Deployment & Distribution

- **PyInstaller Executable**

Freeze the app into a standalone `.exe` for Windows (or `.app` for macOS) so users can run it without Python.

- **Streamlit or Dash Version**

Reimagine the GUI as a web dashboard so users can upload and view results from a browser — ideal for internal teams or cloud deployments.

- **Command-Line Interface (CLI)**

For power users: make a `scorecard_evaluator.py` CLI that runs batch evaluations with arguments like `--file --method xs`.

Innovative Possibilities

- **Report Generator**

After analysis, compile summary, charts, and logs into a PDF report using `reportlab` or `pdfkit`.

- **Machine Learning-based Anomaly Detection**

Offer ML-based optional mode to supplement control charts — e.g. Isolation Forest or One-Class SVM.

- **Data Simulator Tab**

Let users generate their own synthetic SPC data with configurable μ , σ , outlier ratio, and see results live.

Personalization Options

- **Save/Load Sessions**

Let users save their entire GUI state (data, params, plots) as `.json`, and reopen later.

- **Drag & Drop File Input**

Support dragging `.csv` files onto the input box or entire tab.

- **Process Config Presets**

Save named presets like “Line A - Weekday” or “Supplier B - Lot Inspection”.

We have laid a strong and scalable foundation. With these enhancements, our Score Card Evaluator could grow into a full-featured professional toolkit used in QC labs, engineering teams, or teaching environments.

7. References

1. Sheldon M. Ross: "Introduction to Probability and Statistics for Engineers and Scientists", 5th Ed. Academic Press (2014); Douglas C. Montgomery: "Introduction to Statistical Quality Control", 7th Ed. Wiley (2012); Stephen B. Vardeman, J. Marcus Jobe: "Statistical Methods for Quality Assurance: Basics, Measurement, Control, Capability, and Improvement", 2nd Ed. Springer (2016); Irving W. Burr: "Statistical Quality Control Methods", 1st Ed. Marcel Dekker (1976); Acheson J. Duncan: "Quality Control and Industrial Statistics", 5th Ed. Richard D. Irwin (1986); Douglas C. Montgomery: "Design and Analysis of Experiments", 10th Ed. Wiley (2020).
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).
14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).
15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).
16. Volker Ziemann: "Physics and Finance", Springer (2021).

17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).
18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).
35. R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005);

Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);

Software: ccxt-documentation: <https://docs.ccxt.com/#/>, <https://ccxtcn.readthedocs.io/zh-cn/latest/> and <https://pypi.org/project/ccxt-download/>.